



Rapport de Projet
Drone cible
Partie commune
BTS CIEL – option IR

BERNARD GOARANT Timéo

RIBETTE NOLAN

JAUMOUILLE Gatien

2025-2026

1. Introduction	3
1.1 Contexte du projet.....	3
1.2 Objectif du projet.....	3
1.3 Intérêt du projet.....	4
2. Répartition des rôles.....	4
3. Présentation des outils du projet.....	5
3.1 GitHub.....	5
3.2 Google Drive.....	6
3.3 Synthèse.....	6
4. Présentation des outils logiciels.....	6
4.1 Docker.....	6
4.2 ROS2.....	7
5. Schémas de présentation.....	8
5.1 Diagramme d'architecture logicielle et matérielle	8
5.2 Schéma réseau.....	10
5.2.1 Schéma réseau en développement.....	10
5.2.2 Schéma réseau en fonctionnement.....	11
6. Conclusion.....	11
[1] Suivi d'avancement des issues GitHub — vue Burn up.....	12

1.Introduction

1.1 Contexte du projet

Ce projet s'inscrit dans le cadre du BTS CIEL option A au lycée Vauban de Brest, en partenariat avec la Marine nationale et l'IRENAV (Institut de Recherche de l'École Navale). Il répond à un besoin opérationnel réel de la Marine : entraîner les équipages à faire face à des menaces navales modernes.

Depuis plusieurs années, notamment en mer Rouge, des attaques sont menées contre des navires militaires ou civils à l'aide de petits navires rapides (skiffs) ou de drones de surface autonomes (aussi appelé USV : Unmanned Surface Vehicule). Ces menaces sont peu coûteuses, rapides, difficiles à détecter et représentent un risque important.

Dans ce contexte, la Marine nationale souhaite disposer de drones cibles réalistes, capables de simuler ce type d'attaque lors d'exercices. Le projet qui nous est confié consiste donc à concevoir un prototype de drone naval de surface, à faible coût, destiné à être utilisé lors d'entraînements.

1.2 Objectif du projet

L'objectif principal du projet est de concevoir et réaliser un prototype fonctionnel de drone naval de type USV servant de cible d'entraînement pour la Marine nationale.

Plus précisément, le drone doit être capable de :

- Être piloté à distance par un opérateur (mode manuel),
- Suivre un itinéraire autonome à partir de points GPS (mode autonome),
- Se verrouiller sur une cible détectée par caméra et se diriger vers elle (mode verrouillage),
- Fonctionner avec des technologies peu coûteuses et des composants du commerce (approche DIY),
- Être suffisamment fiable pour des essais réels tout en restant économique, car il est conçu pour être potentiellement détruit lors des exercices.

Ce projet sert également de preuve de concept, afin d'évaluer la faisabilité de solutions low-cost pour des applications d'entraînement militaire.

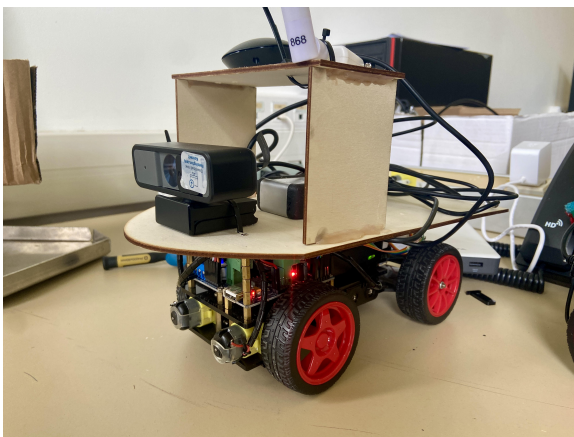


Photo 1 : Photo du prototype du drone cible

1.3 Intérêt du projet

Ce projet présente un triple intérêt : opérationnel, technologique et pédagogique.

- Intérêt opérationnel
 - Il répond à un besoin concret de la Marine nationale en matière d'entraînement face à des menaces modernes.
 - Il permet de simuler des scénarios réalistes sans mobiliser des équipements coûteux.
 - Il contribue à améliorer la préparation et la réactivité des équipages.
- Intérêt technologique
 - Le projet mobilise des technologies actuelles : ROS 2, GPS, vision par caméra, communication LoRa, Docker.
 - Il explore l'utilisation de solutions low-cost dans un contexte militaire, ce qui est stratégique.
- Intérêt pédagogique
 - Il nous permet, étudiants, de travailler sur un projet réel et professionnalisant.
 - Il développe des compétences clés du BTS CIEL : conception système, développement logiciel, communication, tests, documentation et travail en équipe.
 - Il met les étudiants en situation proche de l'ingénierie industrielle et de la recherche appliquée.

2. Répartition des rôles

Le tableau ci-dessous présente la répartition des rôles et des activités individuelles de chaque membre de l'équipe, telle qu'elle a été définie et validée en commission.

Répartition des rôles et des activités individuelles			
Étudiant	Rôle	Mode	Activités principales
BERNARD GOARANT Timéo	Développement Mode Manuel	Manuel	• Développement des nœuds bridge_manette_node et bridge_robot_node • Interface de supervision Node-RED • Intégration Docker mode manuel • Tests et mesures LoRa (portée, RSSI, latence)
JAUMOUILLE Gatien	Développement Mode Autonome	Autonome	• Développement des nœuds IMU et GPS • Algorithme de navigation par waypoints GPS • Fusion des données GPS et IMU
RIBETTE Nolan	Développement Mode Verrouillage	Verrouillage	• Développement du nœud usb_cam • Détection visuelle par caméra (YOLOv5) • Streaming vidéo • Nœud reconnaissance pour la charge vers la cible • Watchdog radio et système

3. Présentation des outils du projet

3.1 GitHub

GitHub est utilisé comme outil central de gestion du projet. Il permet de centraliser l'ensemble du code source, d'assurer le travail collaboratif entre les membres de l'équipe et de conserver l'historique des modifications ainsi qu'un suivi du projet. Le dépôt, nommé `Drone_cible_2026_repo`, est privé et regroupe 4 contributeurs pour un total de 78 commits sur la branche principale `main`. Grâce au système de versions, chaque évolution du projet est tracée et sécurisée, ce qui facilite la correction des erreurs et la validation des développements. GitHub est également utilisé pour héberger la documentation technique, garantissant ainsi la continuité, la fiabilité et la maintenabilité du projet.

Dans le GitHub, nous avons créé une arborescence simple et intuitive.

- SysML : contient les différents diagrammes SysML du projet.
- Simulation : contient les scripts de tests et de simulation.
- Docs technique : l'ensemble des documentations techniques des membres du groupe.
- Code : le code source du projet.

Les fonctionnalités de gestion de projet de GitHub ont été utilisées pour organiser et suivre l'avancement du travail. Les issues permettent de décrire les tâches à réaliser, les anomalies à corriger ou les améliorations à apporter, tout en les attribuant à un membre de l'équipe. Le tableau Kanban offre une vision claire de l'état du projet en classant les tâches selon leur progression (à faire, en cours, terminé). Le travail est découpé en sprints, correspondant à des périodes de développement définies, durant lesquelles des objectifs précis sont fixés. Cette organisation permet un suivi rigoureux, une meilleure répartition des tâches et une adaptation rapide en cas de difficulté.

Le graphique en [annexe 1](#) présente l'évolution du nombre d'issues ouvertes (courbe verte) et fermées (courbe violette) sur l'ensemble de la durée du projet, de janvier à mai 2026. On observe une progression régulière des tâches complétées tout au long du projet. À la date de rendu, le projet totalise environ 34 issues créées pour 25 issues fermées, soit un taux de complétion d'environ 73%. L'écart persistant entre les issues ouvertes et fermées en fin de projet reflète les tâches d'intégration finale et d'assemblage des composants, qui constituent les derniers travaux à réaliser avant la livraison définitive à la Marine nationale.

Le projet a globalement suivi la planification initiale, bien que deux aléas techniques majeurs aient engendré des retards sur certaines parties du développement.

Le premier aléa a été la migration de ROS2 Humble vers ROS2 Jazzy, rendue nécessaire pour assurer la compatibilité avec le logiciel de simulation utilisé par l'équipe. Ce changement de version a impacté l'ensemble des membres du groupe, nécessitant une adaptation des environnements de développement et une vérification de la compatibilité de toutes les dépendances, ce qui a décalé le démarrage effectif des développements de plusieurs semaines.

Le second aléa est lié aux limites de la technologie LoRa pour la transmission de données en continu. Le volume de données télémétriques à transmettre s'est révélé supérieur à ce que le module LA66 peut gérer sans saturation. Des ajustements ont dû être apportés en cours de projet, notamment la réduction de la fréquence de publication des topics ROS2, ce qui a nécessité des phases de tests et d'optimisation supplémentaires.

À la date de remise du rapport, l'état d'avancement est le suivant : le mode manuel est fonctionnel, l'assemblage final des nœuds restant à finaliser ; le mode autonome et le mode verrouillage sont quasiment finalisés. Le prototype est opérationnel sur châssis roulant terrestre, les tests en milieu naval n'ayant pas encore pu être réalisés dans les délais impartis. Cependant les tests naval seront réalisés en simulation très prochainement. Ces éléments constituent les prochaines étapes avant la livraison finale à la Marine nationale.

3.2 Google Drive

Google Drive est utilisé comme espace de stockage et de partage des documents du projet. Il permet de centraliser les fichiers non liés au code, tels que les documents de conception, les comptes rendus de réunion, et les schémas. Cet outil facilite le travail collaboratif en assurant un accès commun aux informations à jour, tout en évitant les doublons et les pertes de données. Google Drive contribue ainsi à une organisation efficace et à la continuité du projet.

Voici son arborescence :

- Compte rendu : contient les comptes rendu des différents sprints.
- Schémas : les différents schémas en lien avec notre projet.
- Soutenance : contient des documents utiles pour la rédaction de la soutenance, le rapport commun ainsi que 3 dossiers :
 - Timéo BERNARD GOARANT
 - Gatien JAUMOUILLE
 - Nolan RIBETTE

Ils contiennent les rédactions de soutenance de chaque membres du groupe.

Google Drive nous sert bien uniquement à déposer des documentations ainsi que des schéma. Vous n'y trouverez en aucun cas du codes dedans.

3.3 Synthèse

Cette organisation en deux outils complémentaires a été choisie pour séparer clairement le développement logiciel de la gestion documentaire : GitHub centralise le code, le suivi des tâches et la collaboration technique, tandis que Google Drive regroupe les documents et schémas du projet. Cette répartition simple garantit un accès rapide à l'information pour chaque membre et réduit les risques de doublons ou d'erreurs.

4. Présentation des outils logiciels

4.1 Docker

Docker est un outil de virtualisation légère basé sur le concept de conteneurs. Contrairement à une machine virtuelle qui émule un système d'exploitation complet, un conteneur Docker partage le noyau du système hôte tout en isolant les processus, les bibliothèques et les dépendances de l'application qu'il contient. Chaque conteneur est défini par une image, construite à partir d'un fichier de configuration appelé `Dockerfile`, qui décrit l'environnement d'exécution nécessaire au bon fonctionnement du logiciel.

Dans le cadre de ce projet, Docker a été utilisé pour garantir la portabilité et l'homogénéité de l'environnement de développement sur les différentes machines de l'équipe et sur la Raspberry

Pi 5 embarquée dans le drone. L'ensemble des nœuds ROS2 ainsi que leurs dépendances sont encapsulés dans des conteneurs, orchestrés via des fichiers `docker-compose.yml`. Cette approche permet de déployer l'intégralité du système logiciel en une seule commande, sans risque de conflit entre les versions des bibliothèques installées.

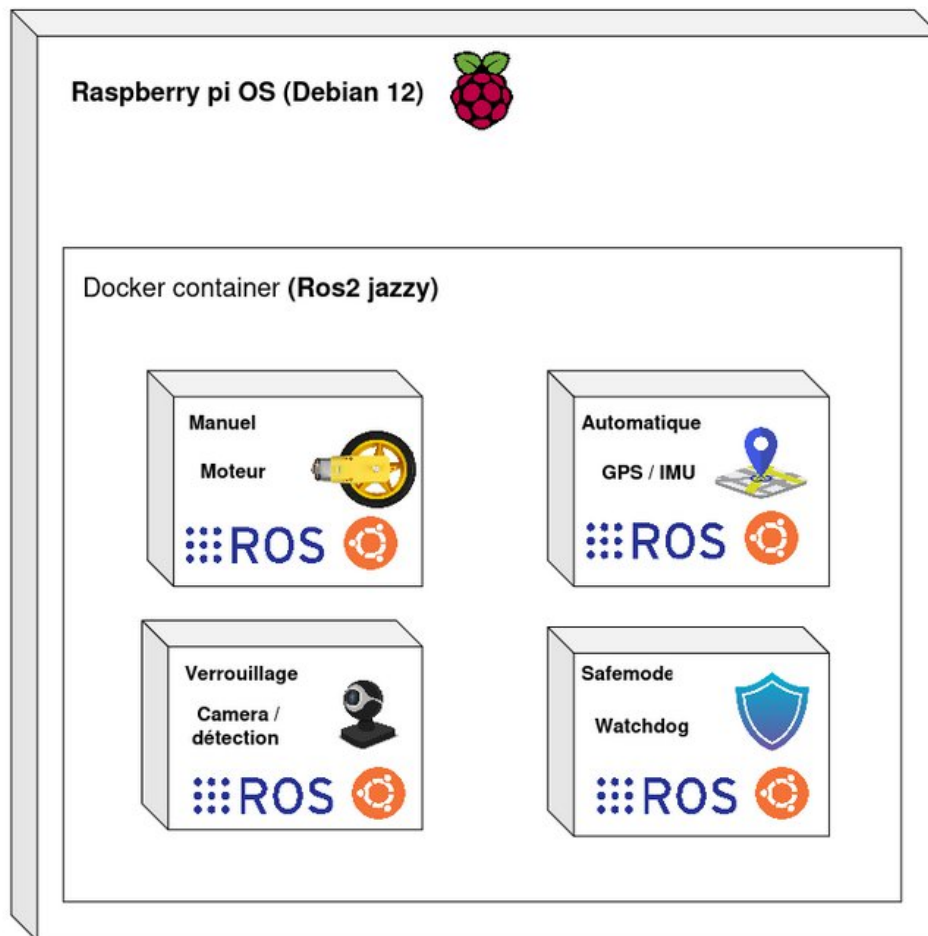


Schéma 1 : Architecture des conteneurs Docker embarqués sur la Raspberry Pi 5

Le schéma 1 illustre l'organisation des conteneurs Docker déployés sur la Raspberry Pi OS (Debian 12). Chaque mode de fonctionnement du drone — Manuel, Automatique, Verrouillage et Safemode — est encapsulé dans un conteneur ROS2 indépendant, regroupant les composants logiciels et les dépendances qui lui sont propres. Cette séparation garantit qu'une défaillance dans un conteneur n'affecte pas les autres modes du système.

Docker assure ainsi un environnement d'exécution stable et isolé pour chacun des composants logiciels du projet. C'est dans cet environnement conteneurisé que s'exécute le framework de robotique ROS2, qui constitue le cœur du système de communication et de contrôle du drone.

4.2 ROS2

ROS2 (Robot Operating System 2) est un framework open source dédié au développement de systèmes robotiques. Il ne s'agit pas d'un système d'exploitation à proprement parler, mais d'un ensemble d'outils, de bibliothèques et de conventions permettant de structurer le code d'un robot sous forme de nœuds communicants. Chaque nœud est un processus indépendant qui remplit une fonction précise — acquisition de données, traitement, commande — et échange

des informations avec les autres nœuds via un système de topics, selon un modèle publish/subscribe.

Par rapport à sa première version, ROS2 apporte des améliorations significatives en termes de fiabilité et de sécurité des communications, grâce à l'utilisation du middleware DDS (Data Distribution Service). Il est également mieux adapté aux systèmes embarqués et aux environnements temps réel, ce qui en fait un choix pertinent pour un projet comme le drone cible.

Dans ce projet, ROS2 est utilisé pour structurer l'ensemble de la chaîne de contrôle. Cette architecture modulaire facilite le développement indépendant de chaque fonctionnalité et simplifie les tests unitaires de chaque nœud.

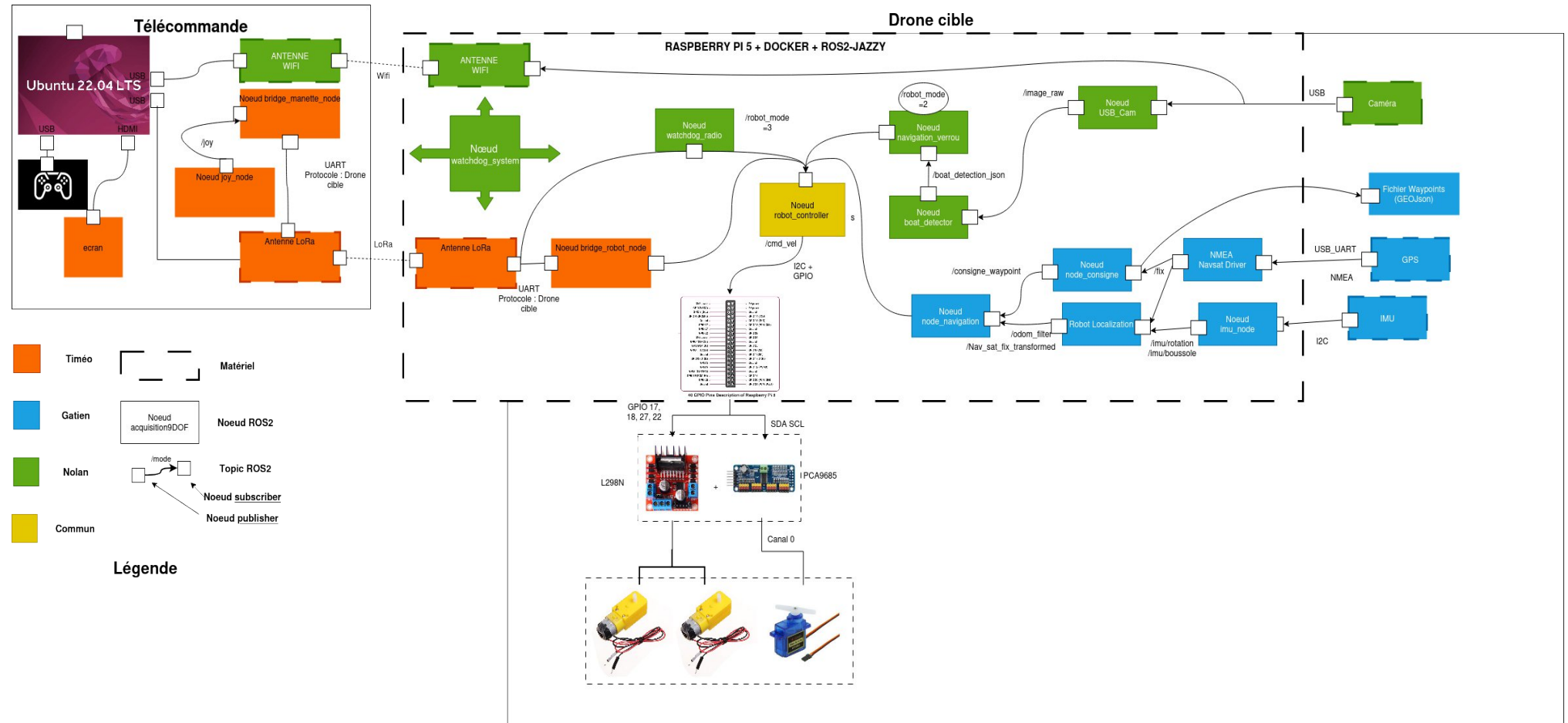
5. Schémas de présentation

Afin de compléter la description technique du projet, cette section regroupe les représentations visuelles de l'infrastructure mise en place. Ces schémas permettent d'appréhender rapidement l'organisation générale du système, aussi bien du point de vue logiciel et matériel que du point de vue réseau.

5.1 Diagramme d'architecture logicielle et matérielle

Le diagramme ci-dessous illustre l'organisation des composants logiciels et matériels du projet, ainsi que les interactions entre chacun d'eux.

Schéma 2 : Diagramme d'architecture globale matérielle et logicielle du système du Drone cible



Ce schéma présente l'architecture complète du système en combinant les aspects matériels et logiciels. Il met en évidence l'organisation du poste opérateur (télécommande), la communication radio ainsi que le système embarqué sur le drone cible. L'ensemble du système repose sur ROS2 exécuté sur la Raspberry Pi 5 avec des conteneurs Docker, permettant de structurer et isoler l'application sous forme de nœuds interconnectés via les topics ROS2. Les différents capteurs du drone (caméra, GPS et IMU) alimentent les modules de navigation et de détection, tandis que les commandes envoyées par l'opérateur sont transmises via une communication radio longue portée utilisant la technologie LoRa. Cette architecture permet de séparer clairement les fonctions de contrôle, de perception et de navigation du système.

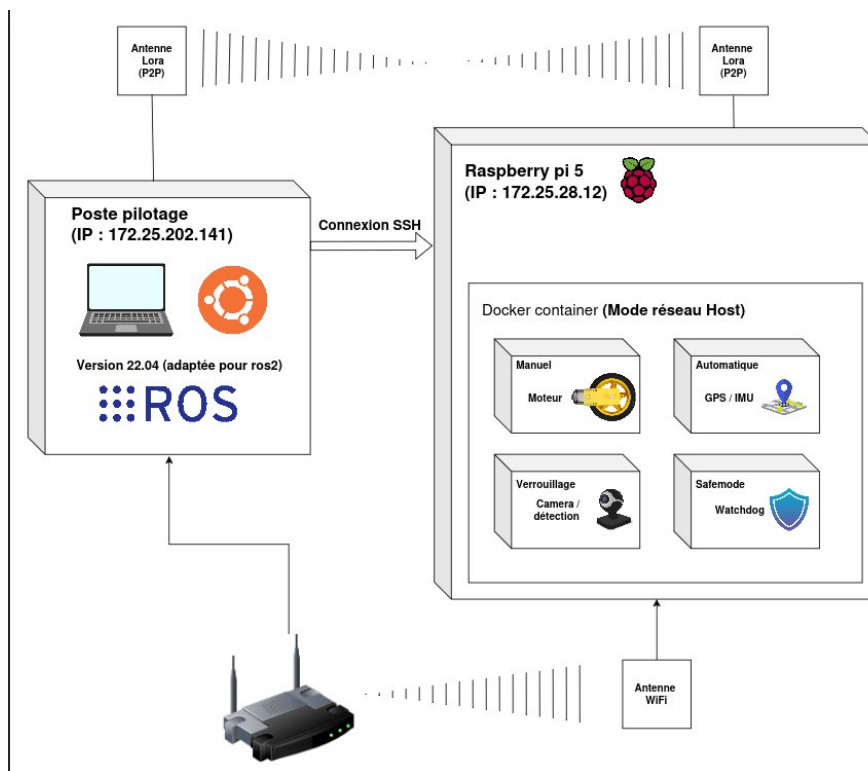
Ce diagramme met en évidence la cohérence de l'architecture retenue et la manière dont les différentes couches du système s'articulent pour répondre aux besoins fonctionnels du projet.

5.2 Schéma réseau

Les schémas réseau suivant représente la topologie de l'infrastructure de communication déployée, en détaillant les équipements impliqués et les flux de données entre eux.

5.2.1 Schéma réseau en développement

Schéma 3 : Schéma architecture réseau en développement



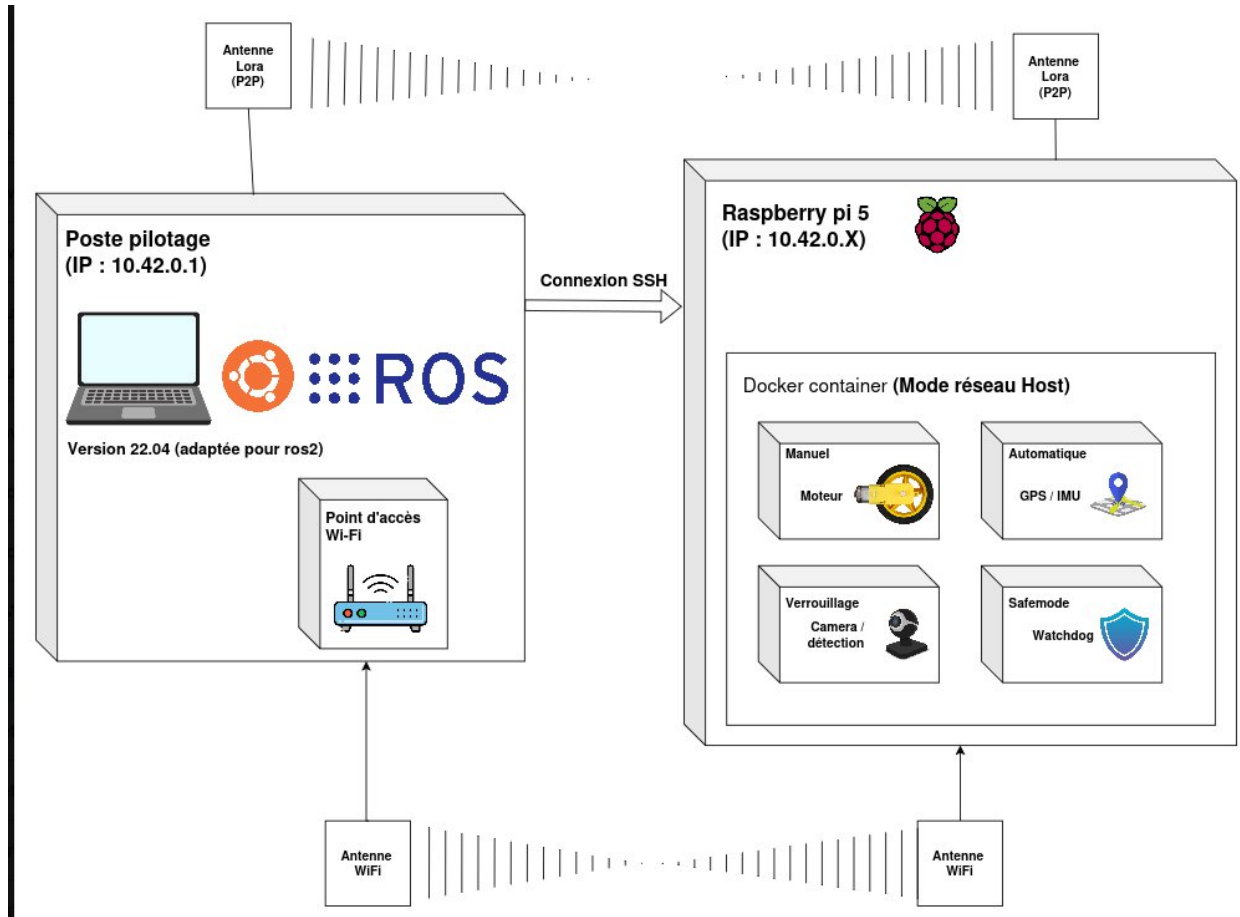
Ce schéma présente l'architecture réseau utilisée pour assurer la communication entre le poste de pilotage et le système embarqué sur la Raspberry Pi 5. Le poste opérateur fonctionne sous Ubuntu 22.04 avec ROS2 installé nativement, permettant le contrôle et la supervision du système. La communication principale entre les deux équipements est réalisée via une liaison radio longue portée LoRa en mode point à point, tandis qu'une connexion Wi-Fi permet l'accès réseau pour la configuration et la maintenance via SSH. Sur la Raspberry Pi, les différents modules fonctionnels sont exécutés dans un conteneur Docker configuré en mode réseau hôte, permettant aux différents services (pilotage manuel, navigation automatique basée sur GPS/IMU, détection par caméra et mécanismes de sécurité) de communiquer efficacement avec le système ROS2 et les

interfaces réseau. Cette architecture assure une séparation claire entre le poste de contrôle et le système embarqué tout en garantissant une communication fiable entre les deux.

Ce schéma permet de visualiser clairement la structure réseau adoptée et de s'assurer que les choix d'architecture garantissent la fiabilité et la sécurité des échanges au sein du système.

5.2.2 Schéma réseau en fonctionnement

Schéma 4 : Schéma architecture réseau en fonctionnement



Ces schémas permettent de visualiser clairement la structure réseau adoptée, en développement et en fonctionnement, et de s'assurer que les choix d'architecture garantissent la fiabilité et la sécurité des échanges au sein du système.

6. Conclusion

Ce rapport commun présente le projet de drone naval de surface développé dans le cadre du BTS CIEL au lycée Vauban de Brest, en partenariat avec la Marine nationale et l'IRENAV. Répondant à un besoin opérationnel concret — simuler des menaces de type USV lors d'exercices d'entraînement — le projet vise à concevoir un prototype fonctionnel, économique et polyvalent, capable d'évoluer en mode manuel, autonome ou de verrouillage sur cible. La gestion du projet repose sur une organisation collaborative structurée, combinant GitHub pour le suivi du code et des tâches, et Google Drive pour la centralisation des documents. L'architecture technique, illustrée par les schémas d'infrastructure logicielle, matérielle et réseau, met en œuvre des technologies modernes telles que ROS 2, Docker, LoRa et la vision par caméra, dans une approche résolument low-cost adaptée au contexte militaire d'entraînement.

Annexe

[1] Suivi d'avancement des issues GitHub — vue Burn up

